

Here, we took  $x$  and  $y$  as inputs, and we defined a `length_tolerance` variable. Remember that some continued fractions are infinite in length, and others are extremely long. By including a `length_tolerance` variable in the function, we can stop our process early if the output is getting unwieldy, and thereby avoid getting caught in an infinite loop.

Remember that when we performed Euclid's algorithm, we used a recursive solution. In this case, we used a `while` loop instead. Recursion is well suited to Euclid's algorithm because it required only one final output number at the very end. Here, however, we want to collect a sequence of numbers in a list. A loop is better suited to that kind of sequential collection.

We can run our new `continued_fraction` generation function as follows:

---

```
print(continued_fraction(105,33,10))
```

---

We'll get the following simple output:

---

```
[3,5,2]
```

---

We can see that the numbers here are the same as the key integers on the right side of [Listing 5-3](#).

We may want to check that a particular continued fraction correctly expresses a number we're interested in. In order to do this, we should define a `get_number()` function that converts a continued fraction to a decimal number, as in [Listing 5-5](#).

---

```
def get_number(continued_fraction):
    index = -1
    number = continued_fraction[index]

    while abs(index) < len(continued_fraction):
        next = continued_fraction[index - 1]
        number = 1/number + next
        index -= 1
    return(number)
```

---

**[Listing 5-5](#)**: *Converting a continued fraction to a decimal representation of a number*